TP 6

Les bases du langage JavaScript

Filière / Niveau	Licence Informatique — 1ère année
Module	Développement Web
Thème 2	JavaScript et Bootstrap
Durée	1 séance (3 h)
Prérequis	Programmation en C (variables, conditions, boucles, fonctions), TP1 à TP5 (HTML, CSS), cycle Git add → commit → push

Objectifs du TP

À l'issue de ce TP, l'étudiant sera capable de :

- Situer **JavaScript** par rapport à HTML et CSS, et repérer ce qui change par rapport au **langage C**.
- Placer un script dans une page : **script interne**, **fichier externe**, attribut defer.
- Utiliser la console du navigateur (y compris pour lire un message d'erreur) et les boîtes de dialogue alert, prompt et confirm.
- Déclarer des variables avec let et const, connaître les **types** et **convertir** une saisie (parseInt, parseFloat, Number, isNaN), distinguer == et === et concaténer avec +.
- Écrire des fonctions (valeurs par défaut, expression de fonction, fonction fléchée).

I. Aperçu théorique

1.1 Pourquoi JavaScript ?

HTML décrit le **contenu** d'une page et CSS sa **présentation**, mais une page HTML/CSS reste **statique** : elle ne sait ni calculer, ni prendre une décision, ni réagir à une action de l'utilisateur autrement qu'en suivant un lien ou en envoyant un formulaire. Les contrôles de saisie du TP3 (required, pattern…) restent limités : impossible, par exemple, de calculer un total ou de comparer deux champs. JavaScript ajoute le **comportement**.

Un langage côté client. Le code JavaScript est téléchargé avec la page, puis exécuté **par le navigateur**, sur la machine de l'utilisateur : il réagit immédiatement, sans attendre le serveur. À l'inverse, un langage côté serveur (PHP, par exemple) s'exécute sur le serveur, avant l'envoi de la page.

1.2 JavaScript et le C : ce qui change

Les conditions, les boucles et les opérateurs s'écrivent presque comme en C : ils ne sont pas repris en détail. Ce tableau résume les différences.

Notion	En C	En JavaScript
Exécution	compilé (gcc), puis exécuté	interprété par le navigateur : pas de compilation, les erreurs apparaissent à l'exécution, dans la console
Point de départ	la fonction main()	pas de main : les instructions s'exécutent de haut en bas au chargement de la page
Déclaration	int n = 5; (type fixé)	let n = 5; (pas de type : il dépend de la valeur et peut changer)
Nombres	int, float, double ; 7 / 2 vaut 3	un seul type number : 7 / 2 vaut 3.5
Texte	char, tableaux de char, strcmp, strcat	type string, entre "…" ou '…' ; comparaison avec ===, concaténation avec +
Booléens	0 = faux, autre valeur = vrai	type boolean : true, false
Entrées / sorties	scanf, printf	prompt, alert, console.log
Fonctions	type de retour, types des paramètres, prototype	mot-clé function, aucun type, pas de prototype

1.3 Où placer le code JavaScript ?

Le code s'écrit dans la balise <script>, de deux façons.

a) Le script interne

Le code est écrit directement entre <script> et </script>. Il est exécuté dès que le navigateur le rencontre, pendant la lecture de la page.

```
<body>
  <h1>Club Informatique</h1>
  <script>
    console.log("Bonjour !");
  </script>
</body>
```

b) Le fichier externe et l'attribut defer

Le code est écrit dans un fichier .js, relié à la page par l'attribut src ; la balise reste **vide**. C'est la bonne pratique : le HTML et le JavaScript sont séparés (comme le CSS au TP4), et un même fichier peut servir à plusieurs pages.

```
<head>
  <meta charset="UTF-8">
  <title>Club Informatique</title>
  <script src="js/app.js" defer></script>
</head>
```

Position du script	Moment de l'exécution
<script> dans le <head>	immédiatement, avant l'affichage du contenu
<script> à la fin du <body>	après la lecture du contenu
<script src="…" defer> dans le <head>	après la lecture de tout le HTML : **méthode retenue dans nos TP**

NB : defer ne s'applique qu'aux fichiers externes (balise avec src).

1.4 La console et les boîtes de dialogue

a) La console du navigateur

Les outils de développement s'ouvrent avec **F12** (ou Ctrl + Maj + I). L'onglet **Console** affiche les messages écrits par console.log et les **erreurs** du script, en rouge, avec le nom du fichier et le numéro de la ligne. On peut aussi y taper une instruction et la valider avec Entrée pour la tester immédiatement.

```
console.log("Membres :", 42);     // affiche : Membres : 42
```

console.log accepte plusieurs valeurs séparées par des virgules : elles s'affichent côte à côte. **À la première erreur, le script s'arrête** : les lignes suivantes ne sont pas exécutées. Si « rien ne se passe », le premier réflexe est d'ouvrir la console.

b) Les boîtes de dialogue

Méthode	Rôle	Valeur renvoyée
alert(message)	affiche un message avec un bouton OK	aucune (undefined)
prompt(message, défaut)	demande une saisie ; défaut (facultatif) pré-remplit le champ	le texte saisi, **toujours une chaîne**, ou null si l'utilisateur clique sur Annuler
confirm(message)	pose une question OK / Annuler	true (OK) ou false (Annuler)
console.log(valeurs)	écrit dans la console (invisible pour l'utilisateur)	aucune

1.5 Variables et types

a) var, let et const

Une variable se déclare **sans type**, avec l'un des trois mots-clés suivants.

Mot-clé	Réaffectation	Portée	Usage
const	non (erreur)	le bloc { }	valeur qui ne change pas : **choix par défaut**
let	oui	le bloc { }	valeur qui change (compteur, total…)
var	oui	toute la fonction	ancien code : **à éviter**

```
const tauxReduction = 50;    // ne changera pas
let total = 0;               // va changer
total = total + 20;
tauxReduction = 30;          // erreur : Assignment to constant variable.
```

Règle : const par défaut, let si la valeur change, jamais var. Les noms s'écrivent en camelCase (nbMembres) et JavaScript distingue les majuscules des minuscules (total et Total sont deux variables différentes).

b) Les types et l'opérateur typeof

Le type d'une variable est celui de la valeur qu'elle contient. L'opérateur typeof renvoie ce type sous forme de chaîne.

Type	Valeurs	Exemple
number	entiers et décimaux (42, 25.5), ainsi que Infinity et NaN (*Not a Number*)	typeof 42 → "number"
string	texte : "Club" ou 'Club'	typeof "42" → "string"

Type	Valeurs	Exemple
boolean	true, false	typeof true → "boolean"
undefined	variable déclarée mais sans valeur	let salle; → "undefined"
null	« aucune valeur », donnée volontairement	typeof null → "object" (erreur historique du langage)

1.6 Les conversions

prompt renvoie toujours une chaîne : "2" n'est pas le nombre 2. Avant tout calcul, il faut **convertir**.

Fonction	Rôle	Exemples
parseInt(ch)	lit un entier au début de la chaîne et s'arrête au premier caractère qui ne convient pas	parseInt("12 ateliers") → 12 parseInt("3.99") → 3 parseInt("abc") → NaN
parseFloat(ch)	même principe avec un nombre décimal ; le séparateur est le **point**	parseFloat("25.5 DT") → 25.5 parseFloat("25,5") → 25
Number(v)	convertit la valeur entière : NaN si elle n'est pas entièrement un nombre	Number("25.5") → 25.5 Number("25.5 DT") → NaN Number("") → 0
isNaN(v)	true si v n'est pas un nombre (après conversion)	isNaN("abc") → true isNaN("12") → false

Attention aux conversions automatiques : avec +, dès qu'un des deux opérandes est une chaîne, JavaScript concatène ("5" + 3 → "53") ; les autres opérateurs convertissent en nombre ("5" * 3 → 15, "5" - 3 → 2). NaN n'est égal à rien, pas même à lui-même : on le teste toujours avec isNaN().

1.7 Les opérateurs

Les opérateurs arithmétiques (+ - * / %), d'affectation (= += -=…), d'incrémentation (++ --) et logiques (&& | | !) s'écrivent comme en C. Les différences :

- Une division par zéro ne provoque pas d'erreur : 10 / 0 vaut Infinity.
- ** calcule une puissance : 2 ** 10 vaut 1024 (en C : pow()).

a) Comparaison : == et ===

Opérateur	Compare	Exemples
== et !=	les valeurs, **après conversion** de type	5 == "5" → true ; 0 == "" → true
=== et !==	les valeurs **et** les types, sans conversion	5 === "5" → false ; 0 === "" → false

Règle : toujours === et !==. Les chaînes se comparent directement (===, <, >), sans strcmp ; la comparaison est alphabétique, caractère par caractère : "10" < "9" vaut donc true.

b) La concaténation

Avec une chaîne, + **concatène** (il remplace strcat et le printf du C). L'expression est évaluée de gauche à droite :

```
const nb = 2;
console.log("Ateliers : " + nb);      // Ateliers : 2
console.log("Total : " + 5 + 3);      // Total : 53
console.log("Total : " + (5 + 3));    // Total : 8
console.log(5 + 3 + " DT");           // 8 DT
let message = "Bonjour";
message += " Sarra";                  // message vaut "Bonjour Sarra"
```

1.8 Conditions et boucles

if / else if / else, switch, l'opérateur ternaire ? :, for, while, do…while, break et continue s'écrivent **exactement comme en C**. Trois différences :

- Toute valeur peut servir de condition. Sont considérées comme fausses : false, 0, "" (chaîne vide), null, undefined et NaN ; tout le reste est vrai, y compris "0".
- switch accepte les **chaînes** (comparées avec ===).
- Le compteur d'une boucle se déclare avec let : dans for (let i = 0; i < 5; i++), i n'existe que dans la boucle.

1.9 Les fonctions

a) Déclaration, paramètres et valeur de retour

```
function calculerTotal(prix, quantite) {
  return prix * quantite;
}
const total = calculerTotal(5, 3);    // 15
```

On n'écrit ni type de retour, ni type des paramètres. Une fonction sans return renvoie undefined. Une fonction déclarée ainsi peut être appelée **avant** sa déclaration dans le fichier : pas besoin de prototype. Un argument manquant vaut undefined ; un argument en trop est ignoré.

b) Valeurs par défaut

```
function saluer(prenom, langue = "fr") {
  return langue === "fr" ? "Bonjour " + prenom : "Hello " + prenom;
}
saluer("Sarra");          // "Bonjour Sarra" : langue vaut "fr"
saluer("Sarra", "en");    // "Hello Sarra"
```

La valeur par défaut est utilisée quand l'argument est absent. Les paramètres qui ont une valeur par défaut se placent en dernier.

c) Portée des variables

Une variable déclarée hors de toute fonction est **globale** : elle est visible dans tout le fichier. Une variable déclarée dans une fonction est **locale** : elle n'existe que pendant l'appel. Avec let et const, la portée se limite même au bloc { } où elles sont déclarées (if, boucle).

```
const club = "Club Informatique";    // globale
function afficher() {
  const salle = "B12";               // locale
  console.log(club, salle);          // Club Informatique B12
}
afficher();
console.log(salle);                  // ReferenceError: salle is not defined
```

d) Expression de fonction et fonction fléchée

En JavaScript, une fonction est une valeur : on peut la ranger dans une variable. Les trois écritures suivantes définissent la même fonction, qui s'appelle toujours de la même façon : carre(4) vaut 16.

Forme	Écriture
Déclaration	function carre(x) { return x * x; }
Expression de fonction	const carre = function (x) { return x * x; };
Fonction fléchée	const carre = (x) => x * x;

Une expression de fonction ou une fonction fléchée ne peut être appelée qu'**après** la ligne qui la définit. Dans une fonction fléchée, un corps réduit à une seule expression est renvoyé sans return ni accolades ; sinon, on écrit les accolades et return :

```
// Sans imbrication : une seule expression (deux écritures au choix)
function carre(x) { return x * x; }   // classique : accolades et return
const carre = (x) => x * x;           // fléchée : ni accolades ni return
console.log(carre(4));                // 16

// Avec imbrication : une condition (deux écritures au choix)
```

```
function mention(note) {          // classique
  if (note >= 10) {
    return "Admis";
  } else {
    return "Ajourné";
  }
}
const mention = (note) => {       // fléchée : accolades et return obligatoires
  if (note >= 10) {
    return "Admis";
  } else {
    return "Ajourné";
  }
};
console.log(mention(12));         // Admis
```

II. Application pratique

Dans ce TP, le dossier et le dépôt s'appellent **TP6**. Le TP comprend trois exercices autour du Club Informatique : un premier script de bienvenue (exercice 1), les types et les conversions (exercice 2) et un quiz (exercice 3). Chaque exercice a sa page HTML et son fichier JavaScript. Les pages HTML ne contiennent qu'un titre : **tout se passe dans la console et dans les boîtes de dialogue**.

Consignes

a. *Le code JavaScript s'écrit dans les fichiers du dossier js ; chaque page relie son script dans le <head>, avec defer.*

b. *La console reste ouverte pendant tout le TP : après chaque modification, enregistrer (Ctrl + S), actualiser (F5) et vérifier qu'aucune erreur rouge n'apparaît.*

c. *Les variables se déclarent avec const, ou let si leur valeur change ; jamais var (sauf à l'étape 2.b, pour l'observer).*

d. *Les comparaisons s'écrivent avec === et !==.*

e. *Tout nombre saisi avec prompt est converti avant d'être utilisé dans un calcul.*

2.1 Préparation et Git (10 min)

- Se placer dans TPWeb, créer le sous-dossier TP6 et le dépôt Git ; sur GitHub, créer un dépôt **vide** nommé TP6 et le relier :

```
mkdir TP6; cd TP6; git init; git branch -M main
git remote add origin <URL_du_dépôt>
```

- Ouvrir le dossier dans VS Code.

2.2 Exercice 1 : Premier script (30 min)

a. **Fichier externe.** Créer bienvenue.html avec le squelette complet (doctype, lang, charset, viewport), le titre « Bienvenue », un h1 « Club Informatique » et un paragraphe « Ouvrez la console (F12) pour suivre le script. ». Créer le dossier js et le fichier js/bienvenue.js, qui :

- ✓ affiche dans la console Bonjour depuis un fichier externe ! ;
- ✓ affiche l'alerte Bienvenue au Club Informatique !.

Relier ce fichier dans le <head> de la page, puis ouvrir la page dans le navigateur, ouvrir la console et actualiser (F5) :

```
<script src="js/bienvenue.js" defer></script>
```

b. Boîtes de dialogue. À la suite, dans bienvenue.js :

- ✓ demander le prénom avec prompt et le ranger dans la constante prenom ; l'afficher dans la console, précédé de Prénom saisi : ;
- ✓ demander avec confirm « Voulez-vous vous inscrire à un atelier ? », ranger la réponse dans inscription et l'afficher dans la console, précédée de Réponse de confirm : ;
- ✓ terminer par l'alerte « À bientôt Ali ! », construite par concaténation avec le prénom saisi.

Tester les cas suivants et compléter le tableau :

Action	Valeur affichée dans la console
Cliquer sur Annuler dans la fenêtre de prompt	
Cliquer sur Annuler dans la fenêtre de confirm	

Que devient l'alerte finale quand on annule la saisie du prénom ?

c. Lire une erreur. Dans bienvenue.js, remplacer volontairement console.log par consol.log sur la ligne qui affiche le prénom, puis actualiser :

- ✓ relever le message rouge de la console et le lien affiché à sa droite (par exemple bienvenue.js:6) ; cliquer sur ce lien : le navigateur montre la ligne fautive ;
- ✓ la fenêtre de confirm et l'alerte finale s'affichent-elles encore ? Pourquoi ? Corriger l'erreur.

Vérification

Avec la saisie « Ali » et OK, la console affiche les trois lignes ci-dessous, sans aucune erreur, et l'alerte finale reprend le prénom.

Console

```
Bonjour depuis un fichier externe !
Prénom saisi : Ali
Réponse de confirm : true
```

Cycle Git : git add . → git commit -m "Exercice 1 : premier script, console et boites de dialogue" → git push -u origin main

2.3 Exercice 2 : Variables, types et conversions (35 min)

Créer types.html (titre « Types », h1 « Types et conversions »), relié au script js/types.js avec defer.

a. Les types. Dans types.js, déclarer les variables suivantes, puis afficher chacune avec son type, sur le modèle de la première ligne :

```
const nomClub = "Club Informatique";
const nbMembres = 42;
const cotisation = 25.5;
const ouvert = true;
let salle;
const responsable = null;

console.log("nomClub :", nomClub, typeof nomClub);
```

Variable	Valeur affichée	typeof
nomClub		
nbMembres		
cotisation		
ouvert		
salle		
responsable		

Quel est le type d'un entier ? D'un décimal ? Le type de responsable vous semble-t-il logique (voir 1.5) ?

b. let, const et var.

✓ Ajouter nbMembres = 43; à la fin du fichier et actualiser. Relever l'erreur, puis la corriger en changeant le mot-clé de la déclaration.

✓ Ajouter le code suivant, actualiser et expliquer les deux résultats. Mettre ensuite la dernière ligne en commentaire (//).

```
if (ouvert) {
  var message1 = "déclarée avec var";
  let message2 = "déclarée avec let";
}
console.log(message1);
console.log(message2);
```

c. prompt renvoie une chaîne. Ajouter les deux lignes suivantes, actualiser et saisir 2. Qu'affiche la console ? Pourquoi ? Ajouter une ligne qui affiche le bon résultat (3) grâce à Number.

```
const saisie = prompt("Combien d'ateliers suivez-vous ?");
console.log("Sans conversion :", saisie + 1);
```

d. **Prévoir, puis vérifier.** Sans rien exécuter, écrire le résultat prévu de chaque expression. Taper ensuite chaque expression dans la console (puis Entrée) et noter le résultat obtenu.

Expression	Prévision	Résultat obtenu
parseInt("3.99")		
Number("")		
"5" + 3		
"5" * 3		
"Total : " + 5 + 3		
5 == "5"		
5 === "5"		
"10" < "9"		

Cycle Git : git add . → git commit -m "Exercice 2 : variables, types et conversions" → git push.

2.4 Exercice 3 : Le quiz du club (45 min)

Créer quiz.html (titre « Quiz », h1 « Quiz du Club Informatique »), relié à js/quiz.js. Le quiz comporte quatre questions et il est noté sur 10.

a. **Déclaration et valeur par défaut.** Écrire la fonction poserQuestion(question, bonneReponse, points = 2), qui :
   - ✓ pose la question avec prompt ;
   - ✓ si la réponse est exactement bonneReponse, affiche l'alerte « Bonne réponse ! +2 points » (avec le nombre de points de la question) et renvoie points ;
   - ✓ sinon, affiche l'alerte « Faux ! La bonne réponse était : a » et renvoie 0.

b. **Expression de fonction.** Écrire calculerPourcentage, une expression de fonction qui reçoit score et total et renvoie le pourcentage. Placer la ligne console.log(calculerPourcentage(7, 10)); **avant** cette définition et relever l'erreur ; la placer ensuite **après** et vérifier qu'elle affiche 70. Pourquoi un appel à poserQuestion placé avant sa déclaration ne provoque-t-il pas d'erreur ?

c. **Fonctions fléchées.** Écrire :
   - ✓ estReussi, en une ligne : elle renvoie true si le pourcentage reçu vaut au moins 50 ;
   - ✓ mention, avec accolades : elle renvoie « Très bien » à partir de 80 %, « Bien » à partir de 50 %, « À revoir » sinon.

d. **Bilan.** Écrire la fonction afficherBilan(joueur, score, total = 10) : elle calcule le pourcentage, affiche les trois lignes du bilan dans la console (voir la vérification) et l'alerte « Sarra : quiz réussi » ou « Sarra : quiz à repasser » (opérateur ternaire et estReussi).

e. **Programme principal.** Demander le prénom du joueur (valeur par défaut « Invité »). Poser ensuite les quatre questions ci-dessous en cumulant le score (+=), puis afficher le bilan.

Question	Bonne réponse	Points
Quelle balise HTML crée un lien ?	a	2 (par défaut)
Quel attribut du label reprend l'id du champ ?	for	2 (par défaut)
Combien vaut 7 / 2 en JavaScript ?	3.5	3
Quel opérateur compare la valeur ET le type ?	===	3

> **Vérification**
>
> Avec le prénom « Sarra » et les réponses a, for, 3.5 et ==, la console affiche le bilan ci-dessous, puis l'alerte « Sarra : quiz réussi ». Actualiser (F5) et recommencer avec les quatre bonnes réponses : 10 / 10, mention « Très bien ».

Console

```
Joueur : Sarra
Score : 7 / 10 (70 %)
Mention : Bien
```

Documentation du dépôt et cycle Git final

Comme aux TP précédents, créer un fichier README.md (un titre et 2 lignes de description) et y répondre, avec vos propres mots, aux deux questions : (1) quand utiliser let, const, et pourquoi éviter var ? (2) quelle est la différence entre == et === (avec un exemple) ?

Cycle Git : git add . → git commit -m "Exercice 3 : fonctions, quiz et README" → git push. Vérifier sur GitHub que l'historique affiche trois commits (un par exercice), et sur votre machine avec git log --oneline.